Console Control

Reference manual — v0.3.10

Show automation for Allen & Heath dLive: discrete cues on a timecode ruler, fired at the console through MIDI Bridge, programmed against your own show recordings.

This is the full manual. The one-pager lives in the Quick Reference.

Table of contents

1. How Console Control thinks
2. The window, top to bottom
3. Timecode sync
4. Tracks and lanes
5. Cues: inserting, selecting, moving
6. Markers, regions and the tempo ruler
7. Reference audio: import and offline conform
8. Names and colours from the console
9. Recording
10. Audio export
11. Loop playback and transport behaviour
12. Show Mode
13. The show file
14. Settings

14a. Licensing and the beta

15. Troubleshooting
16. Acknowledgements

1. How Console Control thinks

Time, not bars. Every cue sits at a timecode position (hh:mm:ss:ff). Live shows drift, drop the click, and vamp — cues locked to the audio survive that; cues locked to an assumed grid do not. Tempo exists in the app, but only to power beat-snapping and the beat grid; the ruler is always time.

Discrete cues, absolute values. A cue is "fader to −6 dB", never "fader down a bit". Whatever happened before it, the console lands in the same state. Continuous cues can carry a transition (duration + curve); everything else is instantaneous.

Chase. When the playhead moves — scrub, locate, play — Console Control works out what every lane's value *should* be at that position and transmits it. That is chase, and it is what makes jumping straight into the second chorus land the console correctly. The global CHASE button suspends all transmission; each lane also has its own CHASE badge (click it to toggle; applies to every selected track).

Fired vs pending. Each lane's readout shows the value that is current at the playhead. Plain text = it was actually transmitted. A dot = this is what the console *should* be at, but it has not been sent (chase off, or the playhead moved without a conform). **Conform Console** transmits everything pending in one pass — a one-shot chase you can use live at any moment.

One console path. Everything reaches the console through the MIDI Bridge's virtual MIDI port. Console Control never opens its own TCP connection — the bridge owns the console's single socket.

2. The window, top to bottom

Header — app name, show name, Edit / Show Mode switch, tier badge.

Transport row — big timecode (click the fps label to change the project frame rate), transport buttons, the **SYNC** button (see §3), chase/console chips, global **CHASE**, **Conform Console**, **Panic**.

Status card — what the app is doing and what that means, in words. When MIDI Bridge turns messages down (it is set to a different console, say), the card reads "MIDI Bridge is refusing messages" with the bridge's own reason and a count, for ten seconds after the last one — the link stays up, but the console doesn't move for those messages.

Toolbar row 1 — Insert strip (see §5), Sync from Console, Preflight, Mixer, Undo/Redo, and the Snap group on the right (the tempo button, Free / Frame / Grid / Beat, note-value selector).

Toolbar row 2 (right) — Marks: + Marker, + Region, marker-lane pills (click to show/hide a lane; colour follows the lane), + Lane, List (opens the editable markers/regions/tempo panel).

Timeline — tracks in folders: Reference audio, Scenes, Actions, Console (grouped by object type). The ruler carries regions, the TEMPO row, marker lanes, the loop band and the time ticks.

Inspector strip — selected cue, time-selection readout, copy/paste, the nudge arrows with their amount selector, lane chase.

Status line — bridge/console state with the IP, and the zoom controls on the right.

The menu bar — **Console Control** (About, Settings… Cmd,, Check for Updates…, Licence…), **File** (shows, import, export), **Edit** (Undo, Redo, snap), **Transport** (play, record, Locate…, Go to Marker / Region…, previous / next marker and cue), **Cue** (the eight inserts), **Console** (Conform, Panic, chase, Sync from Console, Preflight), **Markers**, **Track** (create, lock, solo, arm, previous / next), **View** (Show Mode, the mixer, grid, zoom, track height, waveforms), **Window** and **Help** (Keyboard Shortcuts…, Quick Reference, MIDI Log…, Export Log…, Send Feedback…). A menu shows a command's key when it uses Cmd or Ctrl. Single keys, and keys with only Shift or Opt, work the same but show on the shortcut sheet and in the tooltips instead: as menu keys they would catch letters typed into a dialog.

3. Timecode sync

Sync follows the Reaper model: **the SYNC button arms it, play latches it, stop always disarms it.** With sync armed, pressing play does not move the timeline — it waits for valid code ("Awaiting timecode" in the status bar), latches the moment code locks, follows it while present, freewheels through short dropouts, and goes back to waiting when the code stops. Pressing **stop** drops the latch whether code is present or not; nothing latches again until play is pressed. **Record** with sync armed waits the same way — nothing is written until code locks. With sync off (internal), play and record run freely. A show never latches on its own when it opens.

The **SYNC** button sits next to the record button:

Appearance	Meaning
`LTC/MTC` dimmed	Sync off — internal clock (programming, loop playback)
Lit steel-blue	Armed — press play to latch
Pulsing amber	Play armed, awaiting timecode
Green	Locked to incoming timecode
Amber	Freewheeling — signal lost, coasting

Click toggles sync armed/off (it remembers which type you use). **Right-click** chooses the source on the spot — Internal clock, LTC or MTC, the current one ticked — and its last item, **Sync settings…**, opens Settings on its **Sync** tab: source, the LTC input channel on the audio device, the MTC input MIDI port, and the chase behaviour. A change of source stops the transport, so it is refused in Show Mode. The chase behaviour —

- **Start on valid timecode when stopped** (default off) — the old auto-latch: code alone starts the timeline, no play press needed. Leave it off unless you want a fully unattended rig.

- **Freewheel on missing code** (ms, 0 = forever) — how long a dropout is ridden before the transport pauses and waits again.
- **Re-synchronize if drift exceeds** (ms, 0 = never) — hard relocate threshold.
- **Offset incoming timecode** (ms) — compensates a measured device/converter latency.

While locked to timecode the transport follows the incoming clock: loop playback is suspended and locate/scrub belong to the source.

The wrong frame rate reads as itself, not as a fault: "Timecode is 25 fps — this show is 30 fps" takes the status bar the moment it is seen, ahead of any glitch warning (a code read at the wrong rate breaks continuity once a second, which would otherwise look like a bad cable). Two things can see it — the rate the engine measures, and the rate the code is numbered to — and either is enough. Once the disagreement has held for ten seconds the session log records it (`rate mismatch: code is 25 fps …, project is 30 fps, consistent for 10.0 s`), and again when it clears. The fix is one of two: set the show's rate to match (click the frame rate beside the timecode), or, if the code should be the show's rate, correct the source's sample rate. "Timecode source is running slow" is the second case caught outright, after a few seconds of steady code; a source that stops and comes back at another rate is read afresh, not called slow.

Latency: landing a cue on time

Between a timecode source saying *T* and the console doing what the cue at *T* asks, time is lost in two places: the path the code takes into this Mac (the interface's input buffer, and any converter in front of it such as Time Code Tool turning LTC into MTC), and the path a cue takes out (MIDI to the bridge, the network, the console applying it). Cues themselves fire **to the sample**: each is armed on a timer at its exact time, so there is no tick-to-tick jitter to allow for; what remains is fixed, and two settings on the **Sync** tab remove it:

- **Timecode arrives late by (ms)** — the input path. The transport is placed ahead of the decoded code by this much, so the playhead is where the source is now.
- **Console lead (ms)** — the output path. Every cue is sent this much early, so the console acts as the frame arrives. The MIDI log marks a led cue with the lead it was given.

Calibrate… on the Sync tab sets both for you:

- **Console lead** → **Measure** — ten Get Name queries to the console through MIDI Bridge; the median one-way time plus the console's own 2 ms is the lead. Nothing on the console changes. A spread over 5 ms is flagged: a network that jittery is a show-day risk, not a number to calibrate around.
- **Timecode arrives late by** → **Read device** — what CoreAudio reports for the LTC input device: its latency, safety offset, I/O buffer and stream delay, added up at the device's rate. For MTC through a converter (Time Code Tool), add the converter's own figure.
- **Check by ear** → **Start test** — the proof of the whole chain. Pick a console channel you can hear; a click plays on the reference output on every beat (120 BPM, two bars, looping) while that channel is muted on beats 1 and 5 and unmuted on 3 and 7, sent with the lead like any cue. Listen for the console's mute landing on the click, press **Earlier** or **Later** (2 ms a press, Shift for 10) until it does, then **Keep**. Internal clock only. The timeline plays that loop — the click and the mutes are timed by the transport — but none of the show's own cues are sent while the test runs, and Play does not conform: only the chosen channel's mute goes to the console. Stop restores your loop and leaves the channel unmuted.

Every measurement is written to the session log with its numbers.

Both are per Mac — they belong to the rig, not the show. A third field, **Shift the show against incoming timecode**, is a creative offset that moves the whole timeline (+ = the show runs later) and is unrelated to latency. **Beat snap keeps the beat's exact time** (a beat is rarely on a frame): a cue placed, dragged, moved with others or pasted onto the grid carries its fraction of a frame and fires on it, so it sits on the beat line at any zoom. Frame snap puts a cue back on a whole frame.

Scene recalls are the exception: how long the console takes to recall a scene is the scene's own business and varies, so the lead cannot land one — place a scene cue early by hand if its timing matters.

4. Tracks and lanes

+ Track creates:

- **Console tracks** — pick an object type (input, DCA, aux, matrix, group, FX, main, mute group) and a number or range (`1–16` makes sixteen tracks in one step). Each track carries the lanes its type supports: fader (continuous), mute (state), or both. Colours follow the A&H factory palette.
- **Scenes tracks** — additional scene *categories*. Keep independent scene families (song scenes vs. FX scenes) on separate tracks: on a jump, chase re-fires exactly one scene per track, so unrelated categories never race each other.
- **Actions tracks** — each new one automatically claims the next control number from the action CC pool. The gear on every action lane opens its configuration (action names, control values, and the console-side setup crib).
- **Aux MIDI tracks** — cues that go to something other than the console: a plugin host (Waves SuperRack, LiveProfessor), a lighting desk's MSC input, any MIDI device. See below.

More lanes on a console track — right-click the track:

- **Add send lane…** — a send from the strip to an aux, FX send or matrix, in dB like the fader.
- **Add HPF / EQ lanes…** — on an input channel: **HPF in** (in / out), **HPF frequency** (20 Hz–2 kHz) and each EQ band's **gain** (–15 to +15 dB). Each lane is in its own units: Value… asks for Hz or dB, a frequency ramp sweeps evenly by octave, and the readout shows "120 Hz", "+3.0 dB" or "HPF in". Unity on an EQ gain lane is 0 dB; –∞ and Unity mean nothing on a frequency lane and are refused. An HPF or EQ lane conforms and chases like the fader, but before its first cue a conform leaves the console's EQ as it is. The EQ bands' frequency, width and type come once they are measured on a desk.

Aux MIDI tracks

An Aux MIDI track sends to one **destination**. A destination is either a **virtual MIDI port** that Console Control creates — it appears in the host's MIDI device list as *Console Control - name* while the app is running, and the host remembers it between sessions — or an existing MIDI output (a USB interface, a Network MIDI session, the IAC bus). Each destination has a default MIDI channel and a **host** setting that tunes the editors: SuperRack recalls snapshots past 128 with a bank select before the Program Change; LiveProfessor wants no bank. Destinations are part of the show, so a show carried to another Mac still says where its cues go; a device destination that is not present there is a Preflight failure with a fix. Manage them from an Aux MIDI track's right-click menu (**Aux MIDI destinations…**), where each has a status light and a **Test** button.

Each lane on the track is one of:

- **Single message** — one fixed message; a cue on the lane fires it. A Program Change (with an optional bank), a Control Change with a value, a Note (with velocity and length), or a MIDI Show Control GO / STOP / RESUME with cue and list numbers. SuperRack hot plugins, user keys and snapshots; LiveProfessor cue triggers.
- **Multi-message** — named **states**, each with its own message; a cue picks a state, and chase re-fires the current one, exactly as an Actions lane. Plugin active / bypass; snapshot A / B / C.
- **Variable** — one Control Change the cues ramp. A cue is a value 0–127 (shown as a percent as well), with a ramp time and curve edited exactly as a fader cue's; the value is sent on change while it ramps. At **14-bit** resolution the range is 0–16383, sent as CC *n* then CC *n*+32. SuperRack's plugin knobs; a LiveProfessor hardware-controller fader.
- **Piano** — each cue is a note: its pitch, its own velocity (or the lane's default), and a length on the timeline. The Note On goes at the cue, the Note Off when the length is up — or on Stop, or when the playhead is moved away. Notes are drawn as bars placed by pitch.
- **Send BPM** — follows the tempo map and takes no cues of its own. As **taps**: on entering a tempo (play, a locate, or the map changing under the playhead) the lane's tap message goes on the next beats, as many as you set — a host's Tap Tempo reads the interval; 0 taps means a tap on every beat. As **MIDI clock**: MIDI Beat Clock (24 pulses a beat, with Start and Stop) runs while playing at the tempo under the playhead, which is what LiveProfessor syncs to. Both at once if you like. Where the map has no tempo, a fallback BPM is used.

- **Echo** — follows a console Scenes (or Actions) lane and takes no cues of its own: whenever that lane transmits, the echo lane sends the matching Program Change — scene *n* → PC *n*, plus an offset you set; past 128 the bank select goes with it when the destination's host wants one. Mute the echo track and the console still gets its scene.

Add lanes from the track's right-click menu; the the gear on a lane opens what it sends. **Insert a cue** with the the Insert button in the Insert strip (Opt+A) or by double-clicking the lane (a multi-message lane asks which state). A piano lane also has a **piano editor** (the gear on the lane → Open piano editor…): a piano roll with the lane's notes on pitch rows against the beat grid. Click an empty cell to add a note at the snapped time (length and velocity from the bar above), drag a note to move it in time and pitch, drag its right edge to stretch it, Shift-click to select more, double-click for its pitch, velocity and length, Delete to remove; ↑↓ transpose the selection (Shift an octave), ←→ nudge by the grid. The wheel pans, Cmd-wheel zooms, Opt-wheel scrolls the pitch range. Every edit is an undo step in the main window. Cues appear in the cue list and the Show Mode readout like any other.

No two lanes learn the same thing. A new lane starts on a message no other lane on that destination uses (the next free note or CC); if you set one that is already assigned, the lane's window says so, and Settings → MIDI → **Aux MIDI destinations…** lists every assignment per destination with duplicates marked. Aux MIDI tracks sit under their destination in the track list; **Colour…** on a track's right-click menu sets its colour.

Learning in the host. Both SuperRack and LiveProfessor learn a message from whatever arrives next. Press Learn in the host, then the **the Fire button** on the lane in the track header — it sends the lane's message immediately, outside playback (a multi-message lane pops its states to choose from). The same the Fire button Send buttons are in the lane's the gear window beside each message. A muted Aux MIDI track sends nothing; Stop and Panic send All Notes Off and All Sound Off to every destination.

Tracks are renamed or deleted by **double-clicking the track name** in the panel. The lock icon locks a track against all editing (and against sync); Lock All lives next to + Track. `M` on a console track suppresses its transmission.

Collapse a track with the arrow in its header — collapsed fader lanes still draw their value line with small dB labels. Collapse whole folders from their headers. ↑/↓ steps the selection through visible tracks only.

---

5. Cues: inserting, selecting, moving

The insert strip (buttons enable per selected track):

Button	Cue	Key
`U`	Fader to unity	U
`-∞`	Fader to −∞	I
`V`	Fader to a value…	V
`M` (red)	Mute	M
slashed `M` (green)	Unmute	Shift+M
`S`	Scene recall…	S
`A`	Action…	A

Multi-select tracks (Cmd/Shift-click in the panel) and an insert lands on every selected track in one undo step. The Scene dialog lists all 500 scenes, named where a show file has been imported. Double-click a lane in the timeline for its natural insert; double-click a cue to edit it (value, duration, curve, name, notes).

One selection. However you build it, there is a single set of selected cues:

- **Click** a cue — it becomes the selection.
- **Cmd-click** cues — toggle them in and out.
- **Cmd+A** — every cue on every unlocked track; with a time selection, every cue inside it on its tracks.

While a time selection is being dragged its start, end and length show beside the pointer and in the Range readout.

Tracks — Cmd or Shift-click selects several; **Shift+↑ / Shift+↓** extend the selection a track at a time. Right-click one of several selected tracks → *Delete selected tracks…* removes them in one undo step (a locked track in the selection

refuses the lot).

- **Drag on the timeline background** — the time selection; the cues it covers become the selection. The drag's *vertical* reach picks which tracks are included. **Cmd-drag adds** to the set instead of replacing it.
- **Esc** clears cues + range; a second Esc clears the loop.

Drag any selected cue and the **whole set moves together** — ghost outlines preview every landing position; scene/action cues drag by their small marker (the label text is not a click target, and labels truncate at the next cue's marker). Copy (Cmd+C) / paste-at-playhead (Cmd+V) / delete (Delete) all act on the set.

Where a paste goes. Select another track before pasting and the cues go onto it, each onto the lane with the same parameter (fader to fader, mute to mute, a send to the same send): copy on Input 1, select Input 7, Cmd+V. Drag a new time selection on a track instead and the paste lands at its start, on its tracks. Several tracks copied fill down from the target in the timeline's order. A lane the target hasn't got gets nothing, and the notice says how many cues didn't land. Change nothing and a paste goes back onto the tracks it came from, at the playhead.

Nudge. Opt+←/→ (or the inspector arrows) moves the selection by the amount in the nudge selector: 1/5/10/25 frames, 1 second, or **Beat × div** — one beat-subdivision at the tempo under the first selected cue.

Snap — Free, Frame, **Grid** (the lines you can see, so it scales with zoom), **Beat** (the tempo grid, at the note-value selector's subdivision: whole, half, quarter or eighth notes).

---

6. Markers, regions and the tempo ruler

Markers — Opt+M drops one at the playhead (default lane); Opt+Shift+M asks for a name and lane. Lanes are created from + Lane or inside the marker dialog, shown/hidden from the pills, renamed by double-clicking their label on the ruler. Double-click a marker pill to edit or delete it. Marker text automatically flips dark on light colours.

Regions — Opt+R over the time selection. Drag a region's edge on the ruler to resize; double-click it to rename or delete. The offline conform auto-outlines every decoded segment as a region, and **Markers → Regions from Audio Clips** outlines any reference track's clips on demand.

Tempo — per-span, not global. Detect it from the click track: make a time selection over a steady click section (right-click the click clip → **Select clip's time span** takes the whole song) and press the **tempo** button — the BPM applies to the span where that click clip exists, anchored so the beat grid lands *on* the recorded clicks. A selection over several songs sets one tempo span per song: one per click-track clip it touches, and inside one long recording one per region, each detected on its own. A span with no steady click (the gap between songs) is listed as skipped; the rest are still set. One undo step. A click that changes tempo inside one clip gets a span for each steady stretch, each change on the beat it happens at; a tempo that ramps is followed beat by beat — a short span for every beat of the ramp on the TEMPO row, reported to you as one ramp — so the grid lands on every click. The tempo is fitted through every click in the span (a missed click doesn't bend it) and kept to four decimals; beat 1 sits at the START of the click, to a fraction of a frame — the beat grid is drawn from exactly there, and snapping takes the nearest whole frame, which is where cues live. Detect each song separately; everywhere without a tempo span is free (no beat grid, no beat snap). The TEMPO ruler row shows each span (double-click to edit or free it); the List panel edits them as text. Manual BPM entry covers the current time selection.

Jumping — [and] step the playhead through everything structural: markers on visible lanes, region starts/ends, audio clip starts/ends, and the project start. ← **/** → move it one snap step — a frame, a beat division, or a grid line (Free follows the grid lines on screen). **1 … 9** jump to the first … ninth region on the ruler, **0** to the tenth, **Cmd+1 … Cmd+9** to the eleventh … nineteenth; **G** and **Shift+G** open the list of markers / regions on the nearest one; **T** asks for a timecode; **R** sets the time selection by its two timecodes, filled in with the region under the playhead. The view follows the playhead (the edit cursor, while latched).

---

7. Reference audio: import and offline conform

Import Audio (Cmd+I, File menu, or the + Audio button on the Reference audio folder) accepts up to 12 files. Imported audio is kept inside the show, so a show that has never been saved asks for Save As first, before you pick any files (cancel it and nothing is imported). One file gets the placement dialog (playhead / typed timecode / align by

LTC); several files get the **role dialog** — each file defaults from what it actually contains:

- **LTC (timecode)** — a file that decodes as timecode.
- **Mix with embedded LTC** — music with timecode on one channel; the mix legs are kept, the LTC leg is split off for decoding.
- **Reference mix / Click** — content audio.
- Chunked recordings (`-001`, `-002`, …) are joined automatically.

Offline conform. With an LTC role present the recording is decoded (the show-grade libltc decoder; low-level and DC-biased recordings are handled) and every segment of continuous timecode is placed at its decoded position — one clip per song, sample-aligned, across a Mix track and a Click track. The report shows the segments; **Details** holds drift, gaps and warnings. A frame-rate mismatch offers "change the project and place" in one step. Audio outside the timecode (after LTC stops) is not placed.

Files are copied into the show bundle (48 kHz copied bit-identical, anything else converted honestly) — a show bundle is self-contained.

Waveforms are stereo where files are; Opt-wheel zooms them vertically. Each audio file has a waveform file beside it in the show's media folder (`<file>.peaks`), holding the file's peaks at several resolutions with no limit on length; the timeline draws from the resolution that suits the zoom, and from the file's own samples once you are closer than about five milliseconds a pixel, so an edge is drawn where it is. It is built when audio is imported or recorded, and rebuilt by itself if the audio file changes or the waveform file is missing — it is safe to delete. **Track height** is separate: ↕ – / + in the footer (Ctrl+Opt+↓ / Ctrl+Opt+↑, or Cmd+Opt-wheel over the timeline) scales every track's lanes and waveforms — or only the selected tracks, when two or more are selected; drag a track's bottom edge in the track list to set one track, double-click the edge to put it back, Ctrl+Opt+0 resets, Ctrl+Opt+F fits the tracks to the window. **Waveform size** alone: Ctrl+Opt+= / Ctrl+Opt+– / Ctrl+Opt+9 (or Opt-wheel). Heights and waveform size are view preferences on this Mac — never saved in the show. **View** → **Zoom**: In / Out (Cmd+= / Cmd+–, down to 64 pixels per frame), Zoom to Fit the Show (Cmd+0), Zoom to Selection (Cmd+Shift+0 — the time selection, else the loop, else the region under the playhead). Clips are **locked to the timeline** when placed (the padlock) — timing is what a reference track is for. A locked clip behaves as timeline background: drag across it for a time selection, click to locate. **Right-click a clip** for *Select clip's time span*, *Unlock (allow drag / nudge)* / *Lock to timeline*, *Next take* and *Delete clip…*; an unlocked clip drags: a dashed outline shows where it will land, with the timecode beside the pointer, and several selected clips move together. Click a clip to select it (Cmd or Shift-click for several; Cmd+A takes clips as well as cues); Delete removes the selection after asking. Double-click a clip to retime it by timecode or delete it.

Processing: equaliser, compressor, limiter

Reference audio is there to be heard — a front-of-house mix that wants a little less low end, a click that needs to sit under it, a master that must never clip a monitor. Three processors are built in for that; there is nothing to install and nothing to scan.

```
reference track:  clips → EQ → compressor → fader → pan
master:           sum → EQ → compressor → master fader → limiter
```

Open the Mixer. Under each strip are **EQ** and **CMP**; the master has **LIM** as well. A button is lit when that processor is in. Click one to open the Processing panel on that processor (click it again to close it); **Shift- or Opt-click** switches the processor in or out without opening anything. The panel shows one processor at a time: the tabs along its top choose, and a tab's light is on when that processor is in. Each has its own **IN / OUT** switch, and double-clicking a slider returns it to its default. The compressor and the limiter show three meters: the level going **In**, the **Reduction**, and the level coming **Out**.

- **Equaliser** — a 12 dB/octave high-pass (20 Hz–1 kHz) with its own switch, a low shelf, two bells with Q, and a high shelf; ±18 dB each. The curve drawn is the one applied.
- **Compressor** — an SSL-style bus compressor: stereo-linked, ratio 2:1, 4:1 or 10:1 (the knee is softer at the lower ratios), attack 0.1 to 30 ms, release 0.1 to 1.2 s or **Auto** (quick after a single hit, slow after a passage), make-up gain, a **sidechain high-pass** so a kick does not pump the mix, and **Mix** for parallel compression. It models how that kind of compressor behaves, not one machine's components: it adds no colour of its own. The Reduction bar shows how far it is turning the signal down.

- **Limiter** (master only) — a brick-wall limiter, after the master fader: nothing leaves above its **Ceiling** (–0.3 dBFS by default). **Threshold** is where limiting begins, in dB below the ceiling: the signal is driven up by that much, so at –6 dB the mix is 6 dB louder and anything that would now pass the ceiling is held at it. At 0 dB there is no drive and only what is over the ceiling is touched.

Delay. The equaliser and the compressor add none — not one sample. The limiter has to see a peak coming in order to stop it, so it delays the output by 1.5 ms (72 samples; under a twentieth of a timecode frame) **while it is switched in**, and by nothing when it is out. Switching it is a short fade, not a click.

The settings are part of the show: saved with it, and in every export (section 10). Like the faders, they are not undo steps.

---

8. Names and colours from the console

Two ways in, both matched by console address (type + number), never by name, both writing names and colours only — cues, lanes, track order and bindings are never touched — and both one undo step.

Sync from Console

Console → **Sync from Console** (Ctrl+Opt+R, or the toolbar button) reads every channel's name and colour from the desk through MIDI Bridge — a few hundred quick queries, a few seconds — and shows you what would change before anything does:

- **Names and colours** your tracks will take from the console. A strip that is uncoloured ("off") on the desk leaves your track's colour alone.
- **Console channels that aren't in the show yet** — tick any you want as new tracks (added at the end, with their default lanes and no cues). Nothing is added unless you tick it.
- **Tracks the console no longer has** — listed, never deleted.
- **Locked tracks are left alone** — the lock icon means excluded from sync — and the dialog says how many it skipped.

Apply writes it all as one undo step; **Cancel** changes nothing.

Sync needs MIDI Bridge 1.1 or later running and connected to the console, the transport stopped, and Edit mode (Show Mode refuses it). The console answers every client connected to it, so sync between shows, not during one. A channel that doesn't answer within about two seconds counts as not there — an empty slot, not an error — and a channel named only with its own number is read as the second half of a stereo pair and skipped.

On an SQ, Sync reads the desk's names and colours in one request from MIDI Bridge's own listing (MIDI Bridge 1.4.92 or later, set to the SQ). It sends nothing to the desk's MIDI port; as on the dLive, it needs the transport stopped and Edit mode. A strip still carrying the desk's own name (blank, or a factory name such as ST1 or USB) is left out: it renames nothing and isn't offered as a new track, and a track bound to it isn't listed as gone. A linked stereo pair comes in once, as its first channel. A colour Console Control has no name for comes in as the desk's own.

Console show-file import

File → **Import Console Show File…** reads a dLive show archive (`.tar.gz` from Director's Export → Show or the console's USB export) or a Director CSV export:

- **Scene names** — the archive is the *only* automated source (MIDI-over-TCP has no Get Scene Name, so Sync from Console can't read them). They label the Insert-Scene dropdown, the timeline cue labels and the lane readouts.
- **Channel names + colours** — applied to your existing tracks, the same way as a sync. Both sources give colours; a channel that is coloured on the desk but not named colours its track and leaves your name alone.

Re-import a fresh archive any time; it's the refresh.

---

9. Recording

Arm a reference track (● and pick its input), then Record (Ctrl+Opt+Space): every armed track captures its input into the bundle, stamped at the timeline position where recording started. Takes are named `<Track>-001.wav`, incrementing — a second take never overwrites the first — and appear as clips immediately.

10. Audio export

File → **Export Audio…** (Cmd+E). The window shows only the settings that apply to what you have chosen, and remembers your last choices.

- **What**
  - *Stereo master + LTC* — two files: the master (every audible reference track, summed) and the timecode.
  - *Mono master + LTC* — two files, the master summed to mono.
  - *Mono master and LTC in one file* — one two-channel file, the reference on one side and the timecode on the other: reference left and LTC right, or the other way round. The file any rig that chases can use.
  - *Stereo master only*.
  - *Every reference track* — one file each, with its own gain, pan, equaliser and compressor, and the master fader — but not the master's equaliser, compressor or limiter, which act on the mix and would not do to one track what they did inside it.
- **Format** — **WAV** at 44.1, 48, 88.2 or 96 kHz, in 16-bit (dithered), 24-bit or 32-bit float; or **MP3** at 44.1 or 48 kHz, 128 to 320 kbps, constant bitrate.
- **Span** — the region the playhead is in, the time selection, one region, or **every region containing audio** (one set of files each, named from its region; two regions of one name get `-2`, `-3` …).

The show is 48 kHz. At any other rate the audio is converted (flat to 20 kHz, nothing folded back from above the new rate's limit) and the timecode is **generated at that rate**, not converted — its edges are its own. The timecode is always generated to match the timeline, never copied from a recording.

WAV files are Broadcast WAV: each carries its absolute start timecode, so a BWF-aware DAW drops it in place. MP3 has no such field, so an MP3's start timecode is in its name (`Song-ref+ltc_01000000.mp3`). Timecode survives MP3 at every bitrate offered, and in a two-channel file the channels are coded separately so neither leaks into the other; WAV is still the safe choice for anything that has to chase it.

Renders run the real engine offline — what you hear is what exports, every processor that is switched in included. An export made with the limiter in lines up exactly with one made without it: its delay is taken back out. MP3 is encoded by LAME, a separate library shipped beside the app (see Acknowledgements).

11. Loop playback and transport behaviour

Loop — drag along the ruler's time row (a drag that starts on the existing blue band draws a new loop). Click the blue band to arm/disarm, double-click to clear (or Esc). While playing on the internal clock the transport wraps at the loop end — inside the audio engine, to the sample, with the loop start pre-read, so every pass begins with the loop's first sound and the wrap costs no audio. Loops are for programming; they never fire while locked to timecode.

Scrub — grab the playhead line in the ruler. A plain ruler click locates.

On stop the playhead stays put, or returns to where play started — Settings → "On stop, the playhead".

Play always conforms first (chase permitting), so the console enters playback in the right state.

11a. Cue list mode: firing the show by hand

For a show with no timecode. The timeline stays the programming surface — import a studio or live recording, place the console changes against it — and **Settings** → **Sync** → **This show plays as: Cue list** turns playback into GO presses. The choice is saved with the show, so you can rehearse against the recording in timeline mode and run the night on GO.

Moments. The GO list is read from the timeline, nothing is kept in step by hand. In time order:

- A **marker** groups: every cue from the marker's frame to half a second after it is that marker's moment, named after the marker. Markers on hidden lanes still group.
- A **stack** is one moment: cues on exactly the same time with no marker fire together.
- Everything else is **one cue, one GO**, in timeline order.

GO (Return Return, or Space) fires the standing moment through the ordinary path — a fader ramp runs in real time from the GO — and the transport is placed on the moment, stopped, so readouts and the clock follow; the reference audio does not play. **Back** (Shift+Return, or Shift+Space) stands by on the moment just fired and puts the console back to the state *before* it, so GO fires it again. **Jump to…** (Cmd+Shift+G, or a row in the list) does the same for any moment: the console is put in the state before it, without firing what lies between one by one. ↑ / ↓ in Show Mode move standby without firing. The timeline's **Play** moves to Opt+Space in this mode and plays as ever, for rehearsal; GO is refused while it plays.

Where it shows. Show Mode replaces the countdown with the GO panel: the moment just fired, the standing moment large with its cues underneath, GO and Back, "GO 14 of 63", and the next twenty, each a jump. In Edit mode the toolbar reads "Standby: 14. Chorus of 63" and the Markers & regions list gains a **Cue list** group with the pointer.

Play from GO. Settings → Sync → *On GO* can also play the reference audio from the moment: the transport then runs under the GO for the ear only — no timeline cue fires while it does; the next GO relocates under the next moment and keeps playing; Stop ends it.

Auto-follow. A marker can fire the next moment by itself: double-click the marker (or edit it in the list) and set *auto-follow* in seconds. The GO panel shows "then next after 4 s" on the standing moment and "auto →" while one is counting; any GO, Back, Jump or standby move cancels it.

On a Companion deck the cue list is five actions — GO, Back, Jump to moment (a number), Standby next / previous — and five variables: the playback mode, the standing moment's number and name, the last moment fired, and how many moments the list has. GO, Back and Jump are show-critical, like Stop.

Muted tracks send nothing, as everywhere else. Scenes recalled by GO take as long as the console takes.

12. Show Mode

The locked performance view: huge timecode (green playing, red recording), status chips for transport, sync + lock, chase, bridge health and console-in-sync, live per-track readouts with the fired/pending dot, and Conform Console + Panic. Entering and leaving both ask first; editing and undo are disabled inside. Cmd+Shift+L toggles.

Under the chips is **what is coming**: the region you are in and how long is left of it, then the next moments on the timeline — cues, markers, the start of the next region — each with a countdown and its timecode. Five show at once; scroll the list for up to twenty, and it returns to the top whenever the next moment changes. The first countdown turns amber inside ten seconds and red inside three. Cues on muted tracks and markers on hidden lanes are not listed; several things on one frame are one moment ("Scene 12 +3 more"). In Edit Mode the same readout, cut to a line, sits in the middle of the toolbar: the next moment and the one after it.

13. The show file

A show is a folder bundle:

```
MyShow.showpkg/
  show.json      the model — human-readable, timecode at the boundary
  media/         imported and recorded audio
  calibration/   the fader table in force
  autosave/      rolling autosaves
```

Copy the bundle and everything travels with it; there are no absolute paths. Times are stored as timecode, so changing the project frame rate re-derives every position without moving anything in real time. The loader treats every field as optional — shows from older builds open cleanly, and re-saving upgrades them in place. Autosave runs on

the interval in Settings; closing, New, Open, Open Recent, Close Show and RPP import all warn before discarding unsaved changes.

Save As makes a complete show: the audio the timeline plays (clips and the takes under them), its waveform caches and the calibration files are brought into the new bundle, and the show takes the new file's name. Audio nothing on the timeline uses stays behind. **Close Show** (Cmd+W) puts the show away and leaves an empty one; a closed show is not the one reopened at launch.

Import Reaper Project reads a pack-generated `.rpp` — tracks, cues, markers, regions arrive; referenced audio is listed but not yet copied into the bundle (re-import the WAVs via Import Audio for now).

14. Settings

Console Control → Settings… (Cmd+,) — one window, a tab per section. OK saves every tab, whichever one is showing, and nothing applies before it; a button that opens another window (Calibrate latency…, Licence…) saves as OK does first. The window reopens on the tab you last used.

Tab	Setting	Meaning
Audio	Audio device	Playback/record device (engine restarts between ticks). All of its inputs are opened. A device with only outputs plays but cannot record or chase LTC; one with only inputs plays through the system default output. An interface macOS shows as two devices of one name (inputs on one, outputs on the other, as the M-Audio MobilePre) is listed once and both halves are opened.
MIDI	Bridge MIDI port	The bridge's port name (default `dLive Bridge`)
MIDI	Console	Which console this show is for: dLive or SQ (see below). MIDI Bridge must be set to the same console, or nothing is sent
MIDI	Console base MIDI channel	Matches Utility → Control → MIDI (1–16)
Sync	Clock source	Internal / LTC / MTC
Sync	LTC input channel	Which input carries timecode (1 = first)
Sync	MTC input MIDI port	MIDI port for MTC sync
Sync	Start on valid timecode, Freewheel, Re-synchronize, Offset	The chase behaviour — see Timecode sync
Companion	Bitfocus Companion control	Allowed / Off (presses refused, state still shows)
Companion	Show-critical controls	Allowed / Locked (Stop, Record, Conform, Show Mode refused; Panic always works) — applies to OSC too
Companion	OSC control	Off / This Mac only (127.0.0.1) / Any interface; the UDP port (9000)
Companion	OSC state feed to	host:port that receives /cxc/state/<key>; blank = nobody
Companion	OSC address map…	Every address, generated from the commands
MIDI	Panic recalls scene	1–500, or blank: Panic stops, cancels ramps and quiets Aux MIDI; with a scene named it recalls it too (saved with the show)
Sync	Session start timecode	The timecode of the timeline's first frame (01:00:00:00 for most shows); every timecode shown, typed and chased reads through it

Tab	Setting	Meaning
App	Autosave every	Minutes; 0 disables
App	On stop, the playhead	Stays / returns to play start
App	On launch	New show / reopen the last show
App	Check for updates…	Manual only — see below

Console: dLive, SQ or Wing. A show is for one console, chosen on the MIDI tab and saved with the show; the list holds the consoles Console Control can send to. On an Allen & Heath SQ, Console Control drives inputs 1–48, groups, auxes, matrices, FX sends and returns, LR, DCAs and mute groups (faders and mutes), sends from an input to an aux or FX send and from a group to a matrix, and recalls scenes 1–300. The SQ's MIDI channel and fader law come from MIDI Bridge's console settings, so set them there. Sync from Console reads its names and colours (see Sync from Console). Actions and HPF / EQ lanes are the dLive's, and Measure (latency) isn't available on the SQ yet: enter the console lead by hand, or use Check by ear.

On a Behringer Wing, Console Control drives faders (in dB) and mutes on channels, buses, mains, matrices and DCAs, mute groups, and Scene recalls: what MIDI Bridge has verified on a desk. Aux inputs aren't yet, so an aux input gets no lanes (a test build allows them). A Scene recall needs a show open on the Wing, and changes its whole snapshot. It sends the Wing commands that MIDI Bridge (1.4.103 or later, set to the Wing) turns into the Wing's own OSC: there's no MIDI channel or fader law to set. A fade sends at most one change per screen refresh, and only when the level moves by 0.1 dB. The Wing has no groups: its buses play the aux role. Sends, Actions and HPF / EQ lanes aren't available on the Wing yet; Sync from Console reads its names where MIDI Bridge lists them.

The change of console is refused, naming what's in the way, while the show holds anything the other console hasn't got. When MIDI Bridge is set to a different console from the show, the status card says "Nothing is sent to the console", and Preflight fails.

Check for updates (the Console Control menu, or the button on the App tab) fetches one small file from dylanmaudio.com and compares its version with yours — only when you press it, never at launch and never on a timer, and it is refused while the transport runs, a chase is armed or the show is in Show Mode, so a network call can never touch a show. Nothing is sent but the app's name and version. A beta build checks the beta channel, so testers hear of the next beta; a release build checks the release. "Update available" offers the download page; offline or unreachable just says so, and the app carries on.

Keyboard shortcuts are editable in Help → Keyboard Shortcuts… (Ctrl+Opt+K), persisted per user: click a shortcut and press the keys you want; Delete clears it. One shortcut does one thing: a shortcut another command or a File-menu item already has is marked as you set it and refused on OK, naming the holder (Play and Stop share Space on purpose).

Every dialog closes on Escape or a click outside it, without saving.

How the keys are laid out — single keys are cue programming (U I V M S A, C for chase, L for lock) and getting around (the arrows, [ ], the digits for regions, G, T, R, N). Shift is the pair of a single key (Shift+M unmute, Shift+C chase for everything, Shift+R move the range). Opt is markers and regions (Opt+M, Opt+R), cue-to-cue (Opt+[Opt+]) and nudge. Cmd is the app — files, clipboard, undo, zoom. Ctrl+Opt is the console (conform, sync, preflight, lock all, record) and the view (track height, waveforms, mixer, grid). Return Return is GO and Shift+Return Back in cue list mode (Space does the same there); Opt+A inserts a cue on an Aux MIDI lane. Buttons show their current key when you hover over them. The whole list is in the Quick Reference.

14a. Licensing and the beta

The beta. Every build up to the launch is a beta: it works in full until the date on its badge (top right), and after that it still opens and edits every show but sends nothing to the console. Install the current beta to restore output. Show files are never locked away.

The free tier. Without a licence, Console Control authors five cues of each type — fader, send, mute, mute group, scene, action, aux — and transmits all of them in full, so every message class can be proved on a desk before

buying. The sixth cue of a type is refused with a note; everything else is unlimited.

Licence… is in the Console Control menu and on Settings → App. The panel shows the state — *Licensed to* \<*email*\>*, perpetual* or *rental until* \<*date*\>, with how many of the two Macs the key is on — a key field with **Activate**, **Deactivate this Mac** (frees the activation for another Mac) and **Buy…** (the store, in your browser). Activation is the one network call licensing makes, and like Check for updates it is refused while the transport runs; nothing is ever sent during a show. A licence is read when a show is opened or created, never during a session, so a rental that lapses at midnight does not stop a show that started at nine.

If the licence lapses or cannot be verified (a rental past its date, a key revoked, or no internet for over sixty days), Console Control still opens three more times with output on, each time with a warning naming the count; after that it drops to the free tier with output off until the licence is renewed or verified. Editing is never affected. A show with more cues than the free tier allows then offers three routes on open: reinstate the licence, close the show (the file is untouched), or deactivate and start a blank one.

Every licence event is in the session log — state, tier, Macs — never the key.

New consoles come to a licensed Console Control as free 1.x updates (see Check for updates, below) — the licence is for the app, not for one console family.

14b. OSC control

Any OSC controller — TouchOSC, QLab, a lighting desk, Companion's generic OSC module — can press Console Control's buttons over UDP. Settings → Companion → **OSC control**: *This Mac only* listens on 127.0.0.1 (a controller running on the same Mac), *Any interface* on the network; off by default. One address per command, **/cxc/\<command\>**, with arguments by position where the command takes them (`/cxc/locate "00:01:00:00"`, `/cxc/go-to-marker 3`); **OSC address map…** lists them all, generated from the commands so it is never stale. The same gate as a Companion press applies: Show Mode, locked tracks and the show-critical lock refuse what they refuse, Panic always works, and every message is in the session log.

The **state feed** sends what the deck sees — `/cxc/state/transport`, `timecode`, `lock`, `chase`, `mode`, `go_standby` and the rest — to one host:port, on change only. Leave it blank for nobody.

15. Troubleshooting

macOS 11 (Big Sur). The app runs on macOS 11.0 and later. Its dialogs use the system's WebKit, which gained the dialog element with Safari 15.4; on a Big Sur Mac without that update the app draws them itself, so nothing is lost either way.

"Bridge not running" (red) — start the MIDI Bridge app. Playback keeps time while the bridge is away or reconnecting; cues that fall in that gap are not sent, and the console line says so. Console Control finds its port automatically.

"Bridge up — console disconnected" (amber) — the bridge is fine but has no console: check the console IP in the bridge, the network, and that MIDI-over-TCP is enabled on the console.

SYNC keeps pulsing "awaiting" while timecode plays — wrong input: right-click SYNC → Sync settings… and check the LTC input channel (or the MTC input port), and that the interface actually receives signal on that input. Remember the timeline only listens after play is pressed.

"LTC chase could not start…" on Play — the message says which: the audio device opened with no inputs (an output-only device; pick an interface with inputs in Settings → Audio), or the LTC input chosen on the Sync tab is beyond the inputs the device opened (choose one it has). With "Start on valid timecode when stopped" on, it is reported once and tried again after the next Settings change or device restart.

Conform found no timecode — the file probably isn't the LTC recording; pick the right file/channel in the role dialog. Very quiet or DC-offset recordings are handled automatically.

Cues exist but nothing reaches the console — check the global CHASE button, the track's `M`, the lane's CHASE badge, and the bridge chip. Preflight (Ctrl+Opt+P) runs all of these checks at once and says exactly what is wrong.

A track won't edit — it's locked (the lock icon), or you're in Show Mode.

Paste/insert refused with a tier message — the free tier caps authoring at 5 cues per type. Running a show is never limited.

16. Acknowledgements

Console Control loads two libraries that are not dylanmaudio's. Each is a separate file inside the app (`Console Control.app/Contents/ Frameworks/`), used through its public interface and unmodified:

- **libltc** by Robin Gareus — timecode decoding. LGPL-3.0. https://x42.github.io/libltc/
- **LAME** by the LAME project — MP3 encoding, for audio export. LGPL-2.0 or later. https://lame.sourceforge.io/

Both licences let you replace the library with another build of the same library: swap the file in the Frameworks folder. Their source, as built into this app, is available on request from dylan@dylanmaudio.com.

Independent product — not affiliated with or endorsed by Allen & Heath Ltd. dLive, Avantis, SQ and Qu are trademarks of Allen & Heath Ltd.